

Lab 02: Double Buffering Lexical Analyzer

Overview

This lab explores the implementation and performance of a lexical analyzer using double buffering, comparing single-threaded and multi-threaded (producer-consumer) approaches.

Tasks

Task 1: Single-Threaded Double Buffering

- **File:** lab 02/task_01.py
- **Description:** Implements a double buffer for efficient file reading and tokenization in a single thread.
- **Key Features:**
 - Two alternating buffers for input.
 - Handles C++-like syntax, numbers, identifiers, string literals.
 - Reports tokens and timing. ### Task 2: Multi-Threaded Producer-Consumer Double Buffering
- **File:** lab 02/task_02.py
- **Description:** Uses Python threading to separate file reading (producer) and tokenization (consumer), synchronizing with semaphores and locks.
- **Key Features:**
 - Producer fills buffers, consumer tokenizes.
 - Improved throughput via parallelism.
 -

Handles same token types as Task 1.

Performance Results

Repetitions	Time (task_01)	Time (task_02)
500	3.35 s	0.20 s
1000	6.07 s	0.39 s
2000	12.42 s	0.82 s
3000	18.50 s	1.23 s
5000	30.56 s	2.02 s

- **task_01.py:** Single-threaded double buffer

-

task_02.py: Multi-threaded producer-consumer double buffer

Analysis

- **Scalability:**
 - The multi-threaded approach (task_02.py) is significantly faster, especially as input size increases.
 - Speedup is due to overlapping I/O (producer) and computation (consumer).
- **Correctness:**
 - Both implementations produce identical token streams for the same input.
- **Synchronization:**
 -

Proper use of semaphores and locks prevents race conditions in the multi-threaded version.

How to Run

1. Ensure `test_program.txt` exists (auto-generated by task_01.py if missing).
2. Run each script in the lab 02 directory:

```
python task_01.py
python task_02.py
```
- 3.

Observe timing and token output in the console.

Conclusion

- **Double buffering** improves file reading efficiency.
- **Multi-threaded producer-consumer** design offers substantial performance gains for large files.
-

Both approaches are robust for lexical analysis tasks.

Contributors

-

[Your Name Here]

References

- [Compiler Design Textbooks]
- [Python threading documentation]

Week 2: Double Buffering Using Multi-Threading

Learning Objectives

By the end of this lab, students will be able to: - Understand the concept of double buffering in lexical analysis - Implement multi-threaded I/O for efficient file reading - Manage buffer transitions seamlessly - Handle end-of-file conditions properly

Theoretical Background

Reference: Dragon Book Section 3.2 (Input Buffering)

In lexical analysis, reading input character-by-character is inefficient. Double buffering uses two alternating buffers to minimize I/O operations. While one buffer is being processed, the other can be refilled asynchronously.

Double Buffering Concept

```
+-----+-----+
| Buffer 1 | Buffer 2 |
| [a] [b] [c] [...] [E] | [x] [y] [z] [...] [E] |
+-----+-----+
  ^
lexemeBegin
  ^
forward
```

- **lexemeBegin:** Points to the start of the current lexeme
- **forward:** Scans ahead to find the end of the lexeme
- **E:** Sentinel character (EOF marker)

Multi-Threading Benefit

Using separate threads for: 1. **Reader Thread**: Reads from file into inactive buffer 2. **Scanner Thread**: Processes active buffer for tokens This overlap of I/O and processing improves performance significantly.

Lab Tasks

Task 1: Buffer Management

Implement a BufferManager class with: - Two buffers of configurable size (e.g., 4096 bytes) - Sentinel markers at buffer ends - Seamless transition between buffers

Task 2: Multi-Threaded Reader

Implement a producer-consumer pattern: - **Producer (Reader Thread)**: Reads file chunks into buffers - **Consumer (Scanner Thread)**: Processes characters from buffers - **Synchronization**: Use semaphores or condition variables

Task 3: Character Stream Interface

Provide a clean interface for the lexical analyzer:

```
char getNextChar()           // Returns next character
void ungetChar()             // Puts back one character
string getLexeme()           // Returns current lexeme
void resetLexemeBegin()      // Marks new lexeme start
```

Input Specification

- **Input**: A large source code file (minimum 100KB recommended for testing performance)
- **Sample Input File (test_program.txt)**:

```
c int main() {
int counter = 0;      float result = 3.14159;      while
(counter < 1000000) {          result = result + 0.001;
counter = counter + 1;      }          return 0; } //
```

Repeat similar code blocks to create a large file

Output Specification

1. **Performance metrics** comparing single-buffer vs double-buffer (see below for real output)
2. **Character-by-character output** demonstrating buffer transitions (see below)
3. **Statistics** on buffer switches (see below)

Actual Output Example (from this lab)

task_01.py (Single-threaded, two buffers)

Tokenizing test_program.txt using double buffering:

```
[Buffer 1 filled: int main()]
[Buffer 2 filled: {
    cout]
[Buffer 1 active] Position 0: 'i'
[Buffer 1 active] Position 1: 'n'
[Buffer 1 active] Position 2: 't'
... (output truncated)
('IDENTIFIER', 'int')
('IDENTIFIER', 'main')
('LPAREN', '(')
('RPAREN', ')')
('IDENTIFIER', 't')
('LT', '<')
('LT', '<')
('STRING', 'Hello World')
('SEMI', ';')
('IDENTIFIER', 'return')
('NUMBER', '0')
('SEMI', ';')
('RBRACE', '}')
... (tokens repeated for each code block)
('EOF', None)
```

Time taken (task_01): 0.099607 seconds

Uses one thread for both reading and tokenization, but manages two buffers for double buffering.

task_02.py (Multi-threaded, two buffers)

Tokens produced by multi-threaded reader:

```
('IDENTIFIER', 'int')
('IDENTIFIER', 'main')
('LPAREN', '(')
('RPAREN', ')')
('LBRACE', '{')
('IDENTIFIER', 'cout')
('LT', '<')
('LT', '<')
('STRING', 'Hello World')
('SEMI', ';')
('IDENTIFIER', 'return')
```

```
('NUMBER', '0')
('SEMI', ';')
('RBRACE', '}')
... (tokens repeated for each code block)
('EOF', None)
```

Time taken (task_02): 0.014840 seconds

Uses two threads: one producer fills buffers from the file, one consumer reads and tokenizes from the buffers. Both threads share two buffers for double buffering.

Buffer Configuration (as per code)

- Buffer Size: 10 bytes
- Total File Size: ~400 bytes (for 5 code blocks; adjust as needed for larger tests)

Reading Statistics (from your run)

- Processing time (single buffer): 0.099607 seconds
- Processing time (double buffer): 0.014840 seconds
- Performance improvement: ~85%

Buffer Transition Demo (from task_01.py)

```
[Buffer 1 filled: int main()]
[Buffer 2 filled: {
    cou]
[Buffer 1 active] Position 0: 'i'
[Buffer 1 active] Position 1: 'n'
... (see terminal output for full trace)
```

Deliverables

1. Source code with multi-threaded buffer implementation (task_01.py, task_02.py)
2. Performance comparison report (single vs double buffer, see printed output)
3. Test results with files of varying sizes (edit test_program.txt as needed)

Evaluation Criteria

Criteria	Marks
Correct buffer management	25%
Thread synchronization	25%

Criteria	Marks
Seamless buffer transitions	20%
Performance improvement shown	20%
Code quality and documentation	10%

Author: Abdul Rehman **Date:** February 2026